

Singly Linked List in JavaScript - Complete Guide

I'll show you how to build a fully functional singly linked list in JavaScript, from basic to more advanced operations!

Basic Node Class

```
class Node {  
  constructor(value) {  
    this.value = value; // The data stored in the node  
    this.next = null;   // Pointer to the next node (null if last)  
  }  
}
```

LinkedList Class - Basic Implementation

```
class LinkedList {  
  constructor() {  
    this.head = null; // First node in the list  
    this.tail = null; // Last node in the list (optional but useful)  
    this.length = 0;  // Keep track of list size  
  }  
  
  // Check if list is empty  
  isEmpty() {  
    return this.length === 0;  
  }  
  
  // Get size of list  
  size() {  
    return this.length;  
  }  
}
```

Adding Elements

Append to End (Push)

```
append(value) {  
  const newNode = new Node(value);
```

```

// If list is empty, new node becomes both head and tail
if (!this.head) {
    this.head = newNode;
    this.tail = newNode;
} else {
    // Add to end and update tail
    this.tail.next = newNode;
    this.tail = newNode;
}

this.length++;
return this;
}

```

Prepend to Beginning (Unshift)

```

prepend(value) {
    const newNode = new Node(value);

    if (!this.head) {
        this.head = newNode;
        this.tail = newNode;
    } else {
        newNode.next = this.head;
        this.head = newNode;
    }

    this.length++;
    return this;
}

```

Insert at Specific Position

```

insertAt(index, value) {
    // Check if index is valid
    if (index < 0 || index > this.length) {
        throw new Error('Index out of bounds');
    }

    // Insert at beginning
    if (index === 0) {
        return this.prepend(value);
    }
}

```

```

// Insert at end
if (index === this.length) {
    return this.append(value);
}

// Insert in middle
const newNode = new Node(value);
let current = this.head;

// Traverse to node before insertion point
for (let i = 0; i < index - 1; i++) {
    current = current.next;
}

// Insert new node
newNode.next = current.next;
current.next = newNode;

this.length++;
return this;
}

```



Removing Elements

Remove from Beginning (Shift)

```

removeFirst() {
    if (!this.head) {
        return null; // Empty list
    }

    const removedNode = this.head;

    // If only one node
    if (this.head === this.tail) {
        this.head = null;
        this.tail = null;
    } else {
        this.head = this.head.next;
    }

    this.length--;
    return removedNode.value;
}

```

Remove from End (Pop)

```
removeLast() {
  if (!this.head) {
    return null; // Empty list
  }

  const removedNode = this.tail;

  // If only one node
  if (this.head === this.tail) {
    this.head = null;
    this.tail = null;
    this.length--;
    return removedNode.value;
  }

  // Traverse to second-last node
  let current = this.head;
  while (current.next !== this.tail) {
    current = current.next;
  }

  // Remove last node
  current.next = null;
  this.tail = current;

  this.length--;
  return removedNode.value;
}
```

Remove at Specific Index

```
removeAt(index) {
  // Check if index is valid
  if (index < 0 || index >= this.length) {
    throw new Error('Index out of bounds');
  }

  // Remove first
  if (index === 0) {
    return this.removeFirst();
  }

  // Remove last
```

```

    if (index === this.length - 1) {
        return this.removeLast();
    }

    // Remove from middle
    let current = this.head;

    // Traverse to node before the one to remove
    for (let i = 0; i < index - 1; i++) {
        current = current.next;
    }

    const removedNode = current.next;
    current.next = current.next.next;

    this.length--;
    return removedNode.value;
}

```

Remove by Value (First Occurrence)

```

removeValue(value) {
    if (!this.head) {
        return false; // Empty list
    }

    // Check if head is the target
    if (this.head.value === value) {
        this.removeFirst();
        return true;
    }

    // Search for the value
    let current = this.head;
    while (current.next) {
        if (current.next.value === value) {
            // Found it!
            current.next = current.next.next;

            // Update tail if we removed the last node
            if (!current.next) {
                this.tail = current;
            }

            this.length--;
        }
    }
}

```

```

        return true;
    }
    current = current.next;
}

return false; // Value not found
}

```

Searching and Access

Get Node at Index

```

get(index) {
    if (index < 0 || index >= this.length) {
        return null;
    }

    let current = this.head;
    for (let i = 0; i < index; i++) {
        current = current.next;
    }

    return current.value;
}

```

Find Index of Value

```

indexOf(value) {
    let current = this.head;
    let index = 0;

    while (current) {
        if (current.value === value) {
            return index;
        }
        current = current.next;
        index++;
    }

    return -1; // Not found
}

// Check if value exists
contains(value) {

```

```
    return this.indexOf(value) !== -1;
}
```

Updating Values

```
set(index, value) {
    if (index < 0 || index >= this.length) {
        return false;
    }

    let current = this.head;
    for (let i = 0; i < index; i++) {
        current = current.next;
    }

    current.value = value;
    return true;
}
```

Display and Traversal

```
// Print the list
print() {
    const values = [];
    let current = this.head;

    while (current) {
        values.push(current.value);
        current = current.next;
    }

    console.log(values.join(' → ') + ' → null');
    return values;
}

// Convert to array
toArray() {
    const array = [];
    let current = this.head;

    while (current) {
        array.push(current.value);
        current = current.next;
    }
}
```

```

        return array;
    }

    // Create from array
    fromArray(array) {
        array.forEach(value => this.append(value));
        return this;
    }

```

Advanced Operations

Reverse the List

```

reverse() {
    if (!this.head || !this.head.next) {
        return this; // Empty or single node
    }

    let previous = null;
    let current = this.head;
    let nextNode = null;

    // Update tail first
    this.tail = this.head;

    while (current) {
        nextNode = current.next;
        current.next = previous;
        previous = current;
        current = nextNode;
    }

    this.head = previous;
    return this;
}

```

Find Middle Node

```

findMiddle() {
    if (!this.head) {
        return null;
    }
}

```

```

// Slow and fast pointer technique
let slow = this.head;
let fast = this.head;

while (fast && fast.next) {
    slow = slow.next;
    fast = fast.next.next;
}

return slow.value;
}

```

Detect Cycle (Floyd's Algorithm)

```

hasCycle() {
    if (!this.head) {
        return false;
    }

    let slow = this.head;
    let fast = this.head;

    while (fast && fast.next) {
        slow = slow.next;
        fast = fast.next.next;

        if (slow === fast) {
            return true; // Cycle detected
        }
    }

    return false; // No cycle
}

```



Complete Working Example

```

// Create and use a linked list
const list = new LinkedList();

// Add elements
list.append(10).append(20).append(30);
list.prepend(5);
list.insertAt(2, 15);

```

```

console.log('Original list:');
list.print(); // 5 → 10 → 15 → 20 → 30 → null

// Remove elements
list.removeFirst(); // Removes 5
list.removeLast(); // Removes 30
console.log('\nAfter removals:');
list.print(); // 10 → 15 → 20 → null

// Search
console.log('\nSearching:');
console.log('Index of 15:', list.indexOf(15)); // 1
console.log('Contains 20?', list.contains(20)); // true
console.log('Contains 99?', list.contains(99)); // false

// Update
list.set(1, 25);
console.log('\nAfter update:');
list.print(); // 10 → 25 → 20 → null

// Reverse
list.reverse();
console.log('\nReversed:');
list.print(); // 20 → 25 → 10 → null

// Convert to array
console.log('\nAs array:', list.toArray()); // [20, 25, 10]

```

Interactive Demo Helper

```

// Helper function to demonstrate operations
function demonstrateLinkedList() {
  console.log('🔗 Singly Linked List Demo\n');

  const list = new LinkedList();

  // Build list
  console.log('Building list: append 1,2,3,4,5');
  [1, 2, 3, 4, 5].forEach(n => list.append(n));
  list.print(); // 1 → 2 → 3 → 4 → 5 → null

  // Remove from front
  console.log('\nRemove first (shift):', list.removeFirst());
  list.print(); // 2 → 3 → 4 → 5 → null

```

```

// Remove from end
console.log('\nRemove last (pop):', list.removeLast());
list.print(); // 2 → 3 → 4 → null

// Insert at position
console.log('\nInsert 99 at index 1:');
list.insertAt(1, 99);
list.print(); // 2 → 99 → 3 → 4 → null

// Remove at position
console.log('\nRemove at index 2:', list.removeAt(2));
list.print(); // 2 → 99 → 4 → null

console.log('\nList size:', list.size()); // 3

return list;
}

// Run demo
demonstrateLinkedList();

```



Time Complexity Summary

Operation	Time Complexity	Notes
Append (with tail)	O(1)	Direct access to tail
Append (no tail)	O(n)	Need to traverse
Prepend	O(1)	Just update head
Insert at index	O(n)	Need to traverse
Remove first	O(1)	Just update head
Remove last (with tail)	O(n)	Need to find previous
Remove last (no tail)	O(n)	Need to traverse
Remove at index	O(n)	Need to traverse
Search by value	O(n)	Need to traverse
Get by index	O(n)	Need to traverse
Reverse	O(n)	Visit each node once



Practice Exercises

```

// Exercise 1: Find nth node from end
LinkedList.prototype.nthFromEnd = function(n) {
    // Your code here
    // Hint: Use two pointers, n apart
};

// Exercise 2: Remove duplicates from sorted list
LinkedList.prototype.removeDuplicates = function() {
    // Your code here
};

// Exercise 3: Check if list is palindrome
LinkedList.prototype.isPalindrome = function() {
    // Your code here
    // Hint: Use reverse and compare
};

```

⚠ Common Pitfalls in JavaScript

```

// ❌ Forgetting 'this' context
append(value) {
    // If you use a regular function inside, 'this' changes!
    setTimeout(function() {
        this.append(value); // 'this' is now Window/undefined!
    }, 1000);
}

// ✅ Use arrow functions to preserve 'this'
append(value) {
    setTimeout(() => {
        this.append(value); // 'this' is still the list!
    }, 1000);
}

// ❌ Mutating while traversing
while (current) {
    if (someCondition) {
        current = current.next.next; // Could skip nodes!
    }
    current = current.next;
}

// ✅ Always use a temporary variable or be careful

```



Real-World Use Cases

- **Undo/Redo functionality** in applications
- **Browser history** navigation
- **Music playlist** (next/previous)
- **Image viewer** slideshow
- **Blockchain** (each block links to previous)
- **Hash table collision resolution** (chaining)

This implementation gives you a solid foundation for working with linked lists in JavaScript.

Practice by implementing additional methods like `forEach`, `map`, `filter`, or `reduce` for your linked list!